JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Class ChoiceFormat

java.lang.Object
    java.text.Format
        java.text.NumberFormat
            java.text.ChoiceFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

```
public class ChoiceFormat
extends NumberFormat
```

A `ChoiceFormat` allows you to attach a format to a range of numbers. It is generally used in a `MessageFormat` for handling plurals. The choice is specified with an ascending list of doubles, where each item specifies a half-open interval up to the next item:

```
X matches j if and only if limit[j] ≤ X < limit[j+1]
```

If there is no match, then either the first or last index is used, depending on whether the number (X) is too low or too high. If the limit array is not in ascending order, the results of formatting will be incorrect. ChoiceFormat also accepts `\u221E` as equivalent to infinity(INF).

Note: `ChoiceFormat` differs from the other `Format` classes in that you create a `ChoiceFormat` object with a constructor (not with a `getInstance` style factory method). The factory methods aren't necessary because `ChoiceFormat` doesn't require any complex setup for a given locale. In fact, `ChoiceFormat` doesn't implement any locale specific behavior.

When creating a `ChoiceFormat`, you must specify an array of formats and an array of limits. The length of these arrays must be the same. For example,

- *limits* = {1,2,3,4,5,6,7}
  formats = {"Sun","Mon","Tue","Wed","Thur","Fri","Sat"}
- *limits* = {0, 1, ChoiceFormat.nextDouble(1)}
  formats = {"no files", "one file", "many files"}
  (`nextDouble` can be used to get the next higher double, to make the half-open interval.)

Here is a simple example that shows formatting and parsing:

```
double[] limits = {1,2,3,4,5,6,7};
String[] dayOfWeekNames = {"Sun","Mon","Tue","Wed","Thur","Fri","Sat"};
ChoiceFormat form = new ChoiceFormat(limits, dayOfWeekNames);
ParsePosition status = new ParsePosition(0);
for (double i = 0.0; i <= 8.0; ++i) {
    status.setIndex(0);
    System.out.println(i + " -> " + form.format(i) + " -> "
                             + form.parse(form.format(i),status));
}
```

Here is a more complex example, with a pattern format:

```
double[] filelimits = {0,1,2};
String[] filepart = {"are no files","is one file","are {2} files"};
ChoiceFormat fileform = new ChoiceFormat(filelimits, filepart);
Format[] testFormats = {fileform, null, NumberFormat.getInstance()};
MessageFormat pattform = new MessageFormat("There {0} on {1}");
pattform.setFormats(testFormats);
Object[] testArgs = {null, "ADisk", null};
for (int i = 0; i < 4; ++i) {
    testArgs[0] = new Integer(i);
    testArgs[2] = testArgs[0];
    System.out.println(pattform.format(testArgs));
}
```

Specifying a pattern for ChoiceFormat objects is fairly straightforward. For example:

```
ChoiceFormat fmt = new ChoiceFormat(
     "-1#is negative| 0#is zero or fraction | 1#is one |1.0<is 1+ |2#is two |
System.out.println("Formatter Pattern : " + fmt.toPattern());

System.out.println("Format with -INF : " + fmt.format(Double.NEGATIVE_INFINIT
System.out.println("Format with -1.0 : " + fmt.format(-1.0));
```

```
System.out.println("Format with 0 : " + fmt.format(0));
System.out.println("Format with 0.9 : " + fmt.format(0.9));
System.out.println("Format with 1.0 : " + fmt.format(1));
System.out.println("Format with 1.5 : " + fmt.format(1.5));
System.out.println("Format with 2 : " + fmt.format(2));
System.out.println("Format with 2.1 : " + fmt.format(2.1));
System.out.println("Format with NaN : " + fmt.format(Double.NaN));
System.out.println("Format with +INF : " + fmt.format(Double.POSITIVE_INFINIT
```

And the output result would be like the following:

```
Format with -INF : is negative
Format with -1.0 : is negative
Format with 0 : is zero or fraction
Format with 0.9 : is zero or fraction
Format with 1.0 : is one
Format with 1.5 : is 1+
Format with 2 : is two
Format with 2.1 : is more than 2.
Format with NaN : is negative
Format with +INF : is more than 2.
```

Synchronization

Choice formats are not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

See Also:

DecimalFormat, MessageFormat, Serialized Form

Nested Class Summary

Nested classes/interfaces inherited from class java.text.NumberFormat
NumberFormat.Field

Field Summary

Fields inherited from class java.text.NumberFormat

FRACTION_FIELD, INTEGER_FIELD

Constructor Summary

Constructors

Constructor	Description
ChoiceFormat(double[] limits, String[] formats)	Constructs with the limits and the corresponding formats.
ChoiceFormat(String newPattern)	Constructs with limits and corresponding formats based on the pattern.

Method Summary

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
void	applyPattern(String newPattern)	Sets the pattern.
Object	clone()	Overrides Cloneable
boolean	equals(Object obj)	Equality comparision between two
StringBuffer	format(double number, StringBuffer toAppendTo, FieldPosition status)	Returns pattern with formatted double.
StringBuffer	format(long number, StringBuffer toAppendTo, FieldPosition status)	Specialization of format.
Object[]	getFormats()	

		Get the formats passed in the constructor.
double[]	**getLimits**()	Get the limits passed in the constructor.
int	**hashCode**()	Generates a hash code for the message format object.
static double	**nextDouble**(double d)	Finds the least double greater than d.
static double	**nextDouble**(double d, boolean positive)	Finds the least double greater than d (if positive is true), or the greatest double less than d (if positive is false).
Number	**parse**(**String** text, **ParsePosition** status)	Parses a Number from the input text.
static double	**previousDouble**(double d)	Finds the greatest double less than d.
void	**setChoices**(double[] limits, **String**[] formats)	Set the choices to be used in formatting.
String	**toPattern**()	Gets the pattern.

Methods inherited from class java.text.NumberFormat

format, format, format, getAvailableLocales, getCurrency, getCurrencyInstance, getCurrencyInstance, getInstance, getInstance, getIntegerInstance, getIntegerInstance, getMaximumFractionDigits, getMaximumIntegerDigits, getMinimumFractionDigits, getMinimumIntegerDigits, getNumberInstance, getNumberInstance, getPercentInstance,

getPercentInstance, getRoundingMode, isGroupingUsed, isParseIntegerOnly, parse, parseObject, setCurrency, setGroupingUsed, setMaximumFractionDigits, setMaximumIntegerDigits, setMinimumFractionDigits, setMinimumIntegerDigits, setParseIntegerOnly, setRoundingMode

Methods inherited from class java.text.Format

format, formatToCharacterIterator, parseObject

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

ChoiceFormat

```
public ChoiceFormat(String newPattern)
```

Constructs with limits and corresponding formats based on the pattern.

Parameters:

`newPattern - the new pattern string`

See Also:

applyPattern(java.lang.String)

ChoiceFormat

```
public ChoiceFormat(double[] limits,
                    String[] formats)
```

Constructs with the limits and the corresponding formats.

Parameters:

`limits - limits in ascending order`

`formats - corresponding format strings`

See Also:

setChoices(double[], java.lang.String[])

Method Detail

applyPattern

```
public void applyPattern(String newPattern)
```

Sets the pattern.

Parameters:

`newPattern - See the class description.`

toPattern

```
public String toPattern()
```

Gets the pattern.

Returns:

`the pattern string`

setChoices

```
public void setChoices(double[] limits,
                       String[] formats)
```

Set the choices to be used in formatting.

Parameters:

`limits - contains the top value that you want parsed with that format, and should be in ascending sorted order. When formatting X, the choice will be the i, where limit[i] ≤ X < limit[i+1]. If the limit array is not in ascending order, the results of formatting will be incorrect.`

`formats - are the formats you want to use for each limit. They can be either Format objects or Strings. When formatting with object Y, if the object is a NumberFormat, then ((NumberFormat) Y).format(X) is called. Otherwise Y.toString() is called.`

getLimits

```
public double[] getLimits()
```

Get the limits passed in the constructor.

Returns:

the limits.

getFormats

```
public Object[] getFormats()
```

Get the formats passed in the constructor.

Returns:

the formats.

format

```
public StringBuffer format(long number,
                           StringBuffer toAppendTo,
                           FieldPosition status)
```

Specialization of format. This method really calls `format(double, StringBuffer, FieldPosition)` thus the range of longs that are supported is only equal to the range that can be stored by double. This will never be a practical limitation.

Specified by:

`format` in class `NumberFormat`

Parameters:

`number` - the long number to format

`toAppendTo` - the StringBuffer to which the formatted text is to be appended

`status` - the field position

Returns:

the formatted StringBuffer

See Also:

`Format.format(java.lang.Object)`

format

```
public StringBuffer format(double number,
                           StringBuffer toAppendTo,
                           FieldPosition status)
```

Returns pattern with formatted double.

Specified by:

format in class NumberFormat

Parameters:

number - number to be formatted and substituted.

toAppendTo - where text is appended.

status - ignore no useful status is returned.

Returns:

the formatted StringBuffer

See Also:

Format.format(java.lang.Object)

parse

```
public Number parse(String text,
                    ParsePosition status)
```

Parses a Number from the input text.

Specified by:

parse in class NumberFormat

Parameters:

text - the source text.

status - an input-output parameter. On input, the status.index field indicates the first character of the source text that should be parsed. On exit, if no error occurred, status.index is set to the first unparsed character in the source text. On exit, if an error did occur, status.index is unchanged and status.errorIndex is set to the first index of the character that caused the parse to fail.

Returns:

A Number representing the value of the number parsed.

See Also:

NumberFormat.isParseIntegerOnly(), Format.parseObject(java.lang.String, java.text.ParsePosition)

nextDouble

```
public static final double nextDouble(double d)
```

Finds the least double greater than `d`. If `NaN`, returns same value.

Used to make half-open intervals.

Parameters:

`d - the reference value`

Returns:

`the least double value greather than d`

See Also:

`previousDouble(double)`

previousDouble

```
public static final double previousDouble(double d)
```

Finds the greatest double less than `d`. If `NaN`, returns same value.

Parameters:

`d - the reference value`

Returns:

`the greatest double value less than d`

See Also:

`nextDouble(double)`

clone

```
public Object clone()
```

Overrides Cloneable

Overrides:

`clone` in class `NumberFormat`

Returns:

`a clone of this instance.`

See Also:

`Cloneable`

hashCode

```
public int hashCode()
```

Generates a hash code for the message format object.

Overrides:

`hashCode` in class `NumberFormat`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

equals

```
public boolean equals(Object obj)
```

Equality comparision between two

Overrides:

`equals` in class `NumberFormat`

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is the same as the obj argument; false otherwise.

See Also:

`Object.hashCode()`, `HashMap`

nextDouble

```
public static double nextDouble(double d,
                                boolean positive)
```

Finds the least double greater than `d` (if `positive` is `true`), or the greatest double less than `d` (if `positive` is `false`). If `NaN`, returns same value. Does not affect floating-point flags, provided these member functions do not: Double.longBitsToDouble(long) Double.doubleToLongBits(double) Double.isNaN(double)

Parameters:

d - the reference value

positive - true if the least double is desired; false otherwise

Returns:

the least or greater double value

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.